

Computer Vision

Lecture 8: Object Detection II

Nguyen Manh Toan

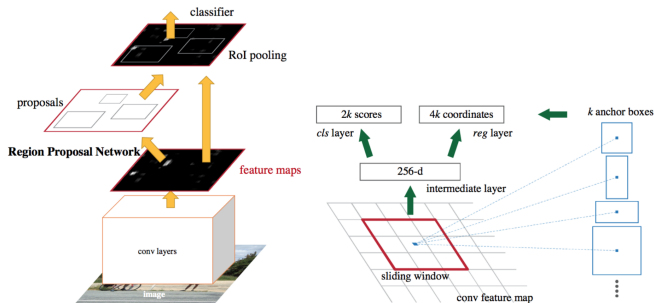
Swinburne Vietnam

Week 8

Where We Left Off

Two-stage detector still has **hand-designed**:

- 1 Anchor scales and aspect ratios
- 2 IoU thresholds for positive and negative
- 3 Sampling ratios
- 4 The NMS threshold



Today we delete them, one at a time, and count what each deletion costs.

The Question To Answer

The proposal stage was doing two jobs at once: it made detection *affordable*, and it quietly protected the classifier from a background it could never have handled directly.

What removing it exposes

A one-stage detector scores every location. Almost all of them are empty — by a factor we will measure in a moment — so the loss becomes almost entirely background, and the gradient follows it.

What we will find

Each component you remove is replaced by something the model learns instead. The bill arrives as data, as training time, or as a new failure mode — and today we can put numbers on all three.

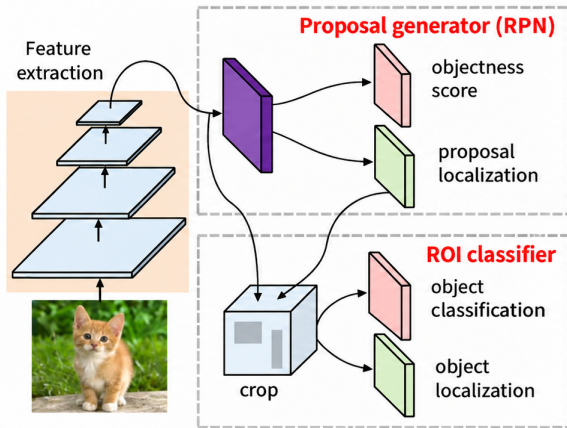
Four hand-designed components. Four deletions.

Today's Lecture

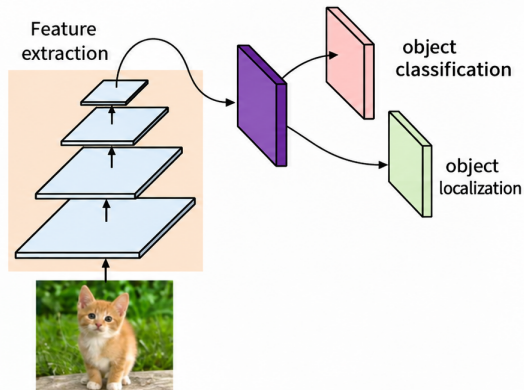
- 1 The Price of Two Stages
- 2 One-Stage Detectors
- 3 Focal Loss: Fixing the Training, Not the Architecture
- 4 Anchor-Free: Deleting the Reference Boxes
- 5 DETR: Detection as Set Prediction
- 6 What You Should Actually Use

One-vs. Two-Stage Detectors

Two-stage detector



One-stage detector



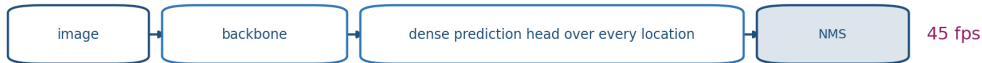
The Price of Two Stages

Where the Budget Goes

Two-stage: the image is looked at twice



One-stage: one pass, one dense head

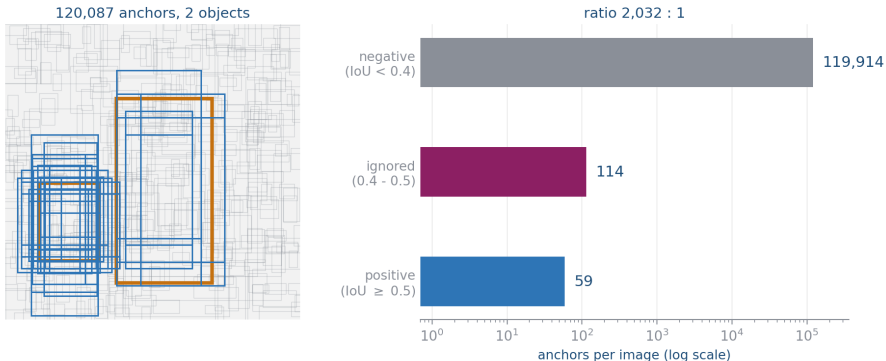


The proposal stage is what you pay for --- and what was protecting the classifier from the background.

The RoI head runs **once per proposal**. Three hundred proposals means three hundred small forward passes, and they cannot be shared.

A one-stage detector replaces that with a convolutional head applied once, everywhere — which is the only way to reach real-time.

The Imbalance



Computed for a RetinaNet anchor set (P3--P7, 9 per location, 800×800). A two-stage detector never shows its classifier this distribution; a one-stage one must.

The Obvious One-Stage Detector Fails

What goes wrong

With ≈ 2000 background anchors for every object, the summed cross-entropy is dominated by examples the model *already gets right*. The gradient points at “keep predicting background”.

Why two-stage detectors escaped it

The RPN threw away almost all of the background before the expensive head ever ran, and then the head sampled a fixed 1:3 ratio. Neither protection exists now.

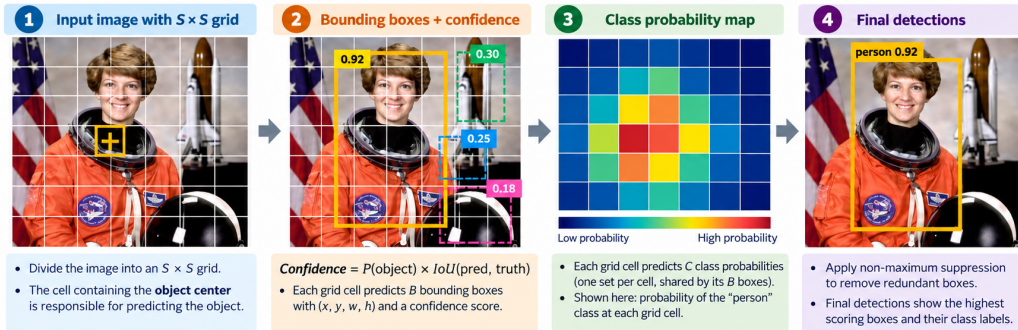
Two ways out

Sample the negatives (SSD’s hard negative mining), or **reweight** them (RetinaNet’s focal loss). The second turned out to be the better idea, and we will see exactly why.

One-Stage Detectors

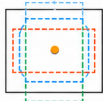
YOLO (Redmon et al., 2016): You Only Look Once

One forward pass. Predict bounding boxes, confidence scores, and class probabilities for each grid cell.



What does each grid cell predict?

One grid cell



B anchor boxes
(per cell)

For each of B boxes:

x	y	w	h	conf
-----	-----	-----	-----	---------------

(5 numbers per box)

C class probabilities

(one set per cell, shared by all boxes)

Total output tensor

Each cell emits one vector of length:

$B \times 5 + C$

B boxes
(x, y, w, h, conf)

C class
probabilities

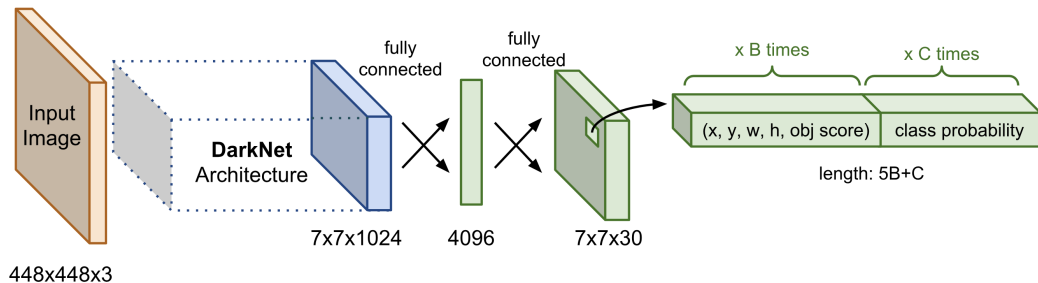
Total prediction tensor for one image:

$S \times S \times (B \times 5 + C)$

Key points

- One forward pass produces all predictions.
- The image is divided into an $S \times S$ grid.
- Each grid cell predicts B bounding boxes, confidence scores, and C class probabilities.
- The cell containing the **object center** is responsible for predicting the object.
- Total output tensor: $S \times S \times (B \times 5 + C)$.

YOLO Architecture



The architecture has 24 convolutional layers and 2 fully connected layers, pretrained on ImageNet at 224×224 and fine-tuned for detection at 448×448 .

The base model is similar to GoogLeNet with inception module replaced by 1×1 and 3×3 conv layers. The final prediction of shape $S \times S \times (5B + C)$ is produced by two fully connected layers over the whole conv feature map.

Localization loss for bounding box offset prediction

$$\mathcal{L}_{\text{loc}} = \lambda_{\text{coord}} \sum_{i=1}^{S^2} \sum_{j=1}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

Classification loss for conditional class probabilities

$$\mathcal{L}_{\text{cls}} = \sum_{i=1}^{S^2} \sum_{j=1}^B (\mathbb{1}_{ij}^{\text{obj}} + \lambda_{\text{noobj}}(1 - \mathbb{1}_{ij}^{\text{obj}}))(C_{ij} - \hat{C}_{ij})^2 + \sum_{i=1}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \mathcal{C}} (p_i(c) - \hat{p}_i(c))^2$$

- $\mathbb{1}_{ij}^{\text{obj}}$ indicates whether the j-th bounding box of the cell i is “responsible” for the object prediction
- $\mathbb{1}_i^{\text{obj}}$ indicates whether the cell i contains an object

YOLO loss:

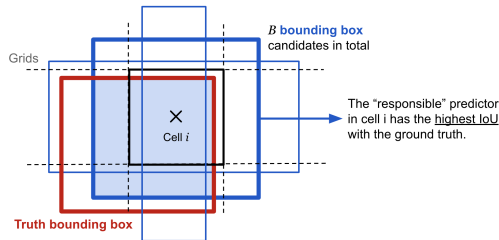
$$\mathcal{L} = \mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{cls}}$$

What The YOLO Loss Reveals

One sum-squared loss over the whole output tensor:

- $\lambda_{\text{coord}} = 5$ — the coordinate terms matter more than the rest
- $\lambda_{\text{noobj}} = 0.5$ — confidence on empty cells matters *less*, because most cells are empty
- Regress $\sqrt{w}$, $\sqrt{h}$, so a fixed error costs more on a small box than a large one

This is the imbalance problem, patched with a constant chosen by hand. Focal loss is the principled version of this line.



Determine the responsible grid cell: the cell i containing the center of the ground-truth object. Then, among the B bounding boxes predicted by that cell, choose the one whose current prediction has the highest IoU with the ground-truth box,

Model	VOC07 mAP	FPS	Note
Faster R-CNN (VGG-16)	73.2	7	Lecture 7's end point
YOLO	63.4	45	real time, on 2016 hardware
Fast YOLO	52.7	155	9 conv layers

The interesting error analysis

YOLO makes **more localisation errors** than Fast R-CNN, but far fewer background false positives: 13.6% of Fast R-CNN's top detections are background, and it is nearly $3\times$ more likely than YOLO to hallucinate one.

Why

YOLO sees the *whole image* when it predicts. A region-based classifier sees a crop, and a crop of background can look like anything. Context is free here, and expensive there.

The other side of the one-class-per-cell design: YOLO v1 is weak on small objects appearing in groups, because a single cell cannot name two things.

Single-Shot Detector —SSD (Liu et al., 2016)

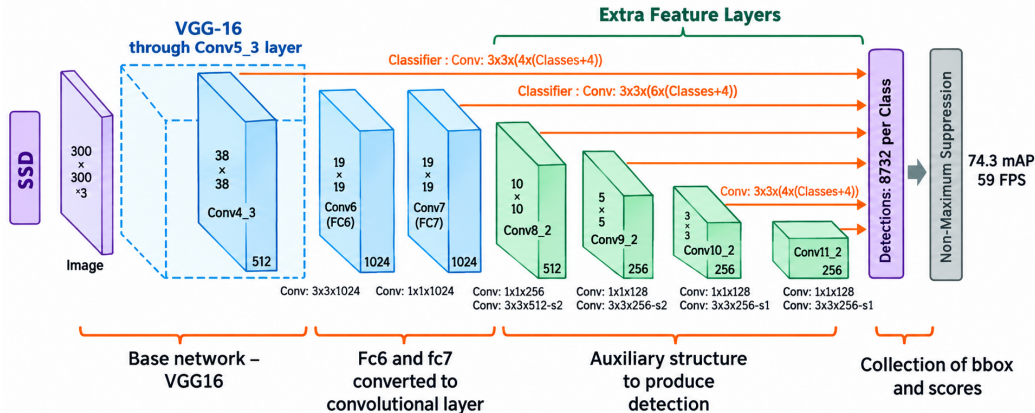
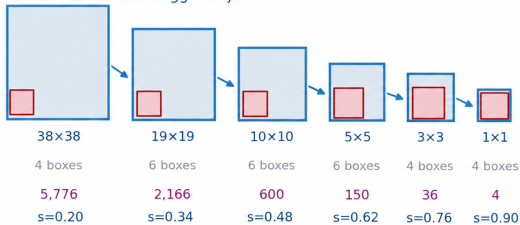


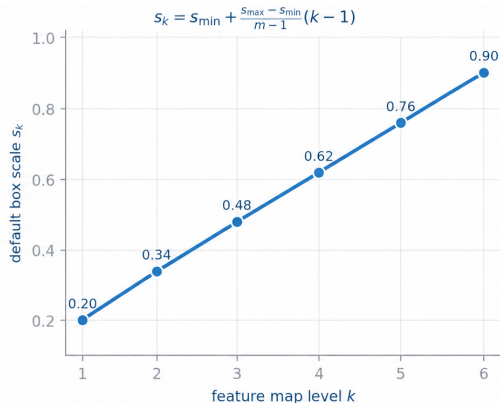
Figure: SSD300 with VGG-16 backbone network through Conv5_3 layer. A few extra feature extraction layers are also added.

Defaults at Every Scale

Six feature maps predict at once: deeper is coarser, and coarser means bigger objects



8,732 default boxes per image, all scored in one pass



The anchor boxes on different levels are rescaled so that one feature map is only responsible for objects at one particular scale

Anchor Boxes

Default boxes, similar to anchor boxes in Faster RCNN. Feature maps are divided into grids. Each grid associates itself with 4 default boxes of different aspect ratios. For each of the default boxes, the SSD model predicts box shape offsets and confidence scores for each category.

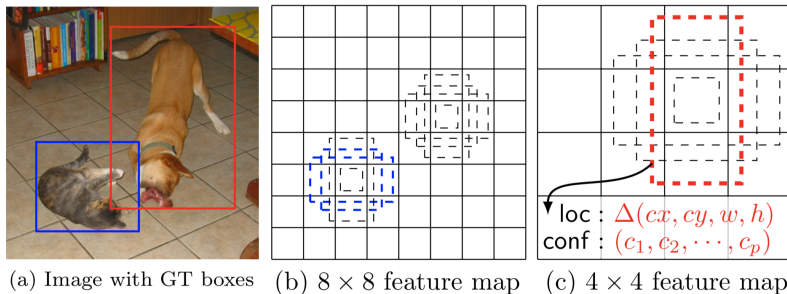


Figure: In a fine-grained feature map (8×8), the anchor boxes correspond to a smaller area of the raw input. In a coarse-grained feature map (4×4), the anchor boxes cover a larger area of the raw input.

What SSD Added

Multi-scale prediction

Detect from six feature maps rather than one. This is the “pyramidal feature hierarchy” of Lecture 7 — the version FPN would later improve by making the shallow levels semantically strong.

Hard negative mining

Sort the negatives by confidence loss, keep the worst, and hold the negative-to-positive ratio at most 3:1.

A *selection* rule: most negatives contribute nothing because they are discarded.

Model	VOC07 mAP	COCO AP	FPS
SSD300	74.3	23.2	59
SSD512	76.9	26.8	22

SSD300 beats Faster R-CNN on VOC while running $8\times$ faster — the first time a one-stage detector was not simply the cheap option.

The First Generation, Honestly

What they proved

- Proposals are not necessary for accuracy on VOC
- Dense prediction is the only route to real time
- Seeing the whole image suppresses background errors

What they did not solve

- On **COCO**, both trailed two-stage detectors badly
- Both handled the imbalance with a hand-set constant or a selection rule
- Anchors, IoU thresholds and NMS all survived untouched

The open question of 2017

Is a one-stage detector *intrinsically* less accurate, or was it just being trained wrong?

Focal Loss: Fixing the Training, Not the Architecture

The Diagnosis

Lin et al. (2017) argued the gap was never architectural, but **class imbalance during training**.

An easy negative contributes a small loss — but there are 10^5 of them and 10^1 objects, so small $\times$ enormous still wins.

The move

Do not select, do not resample. Keep **every** anchor in the sum and down-weight the ones the model already handles.

$$\text{CE}(p_t) = -\log p_t \quad \longrightarrow$$

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log p_t$$

p_t is the probability assigned to the *true* class. The paper uses $\gamma = 2$, $\alpha = 0.25$.

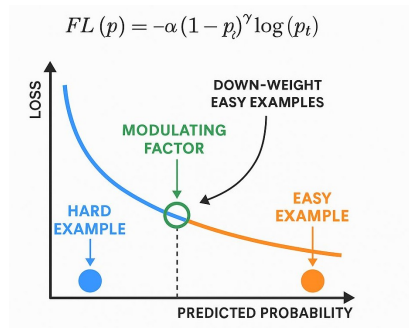
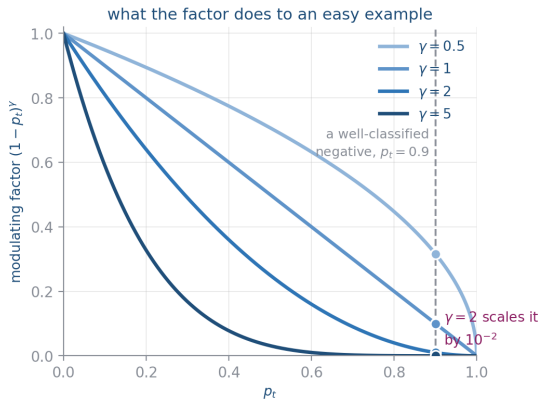
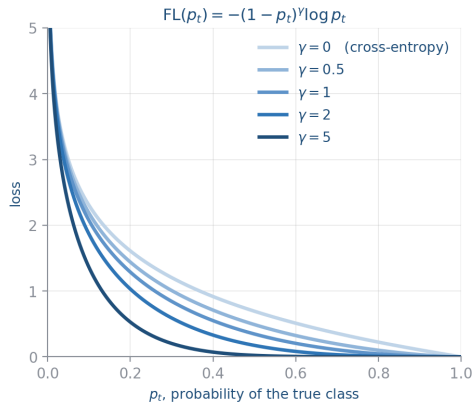


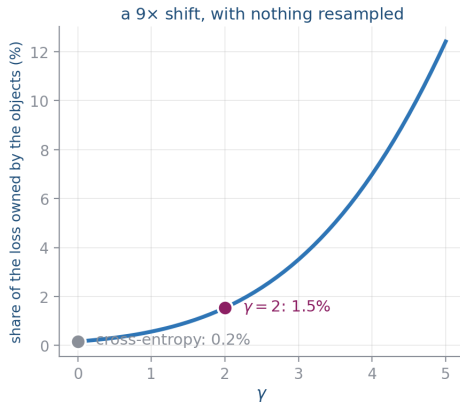
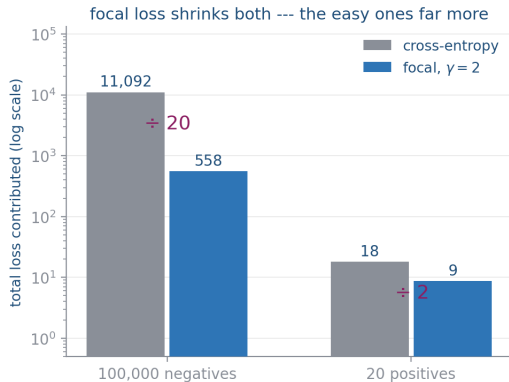
Figure: Focal loss focuses on hard examples.

What the Modulating Factor Does



At $\gamma = 2$, an anchor the model already scores at $p_t = 0.9$ contributes $1/100$ of what it did under cross-entropy — while an anchor at $p_t = 0.1$ is barely touched. Nothing is discarded; the sum is simply re-weighted towards what is still wrong.

Where the Gradient Actually Goes



Simulated: 20 positives and 100,000 negatives from a partly-trained detector. No mining, no fixed ratio --- only the loss is reweighted.

Two Details That Matter

Why $\alpha = 0.25$, not higher

α_t weights the classes directly: α for foreground, $1 - \alpha$ for background. On its own you would raise it. But γ has *already* shifted the weight towards the hard examples — and those are mostly negatives — so α swings back to compensate.

The two interact. Tuning them separately gives the wrong answer for each.

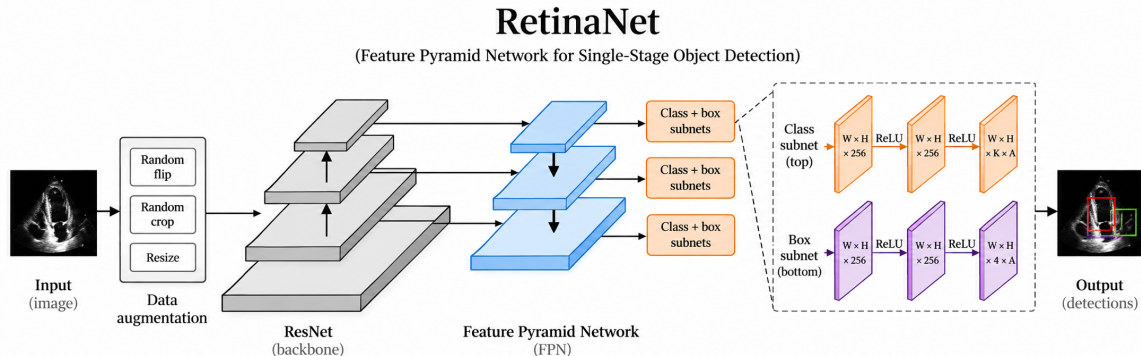
The initialisation trick

Initialise the classification bias so the model starts by predicting $p \approx \pi = 0.01$ for every anchor:

$$b = -\log\left(\frac{1 - \pi}{\pi}\right)$$

Without it, a randomly initialised dense head emits 10^5 confident foreground scores in the first iteration and the loss diverges.

The paper is explicit that training was *unstable* without this line. A loss function and its initialisation are not separable.



RetinaNet has $k = 9$ anchors per location. The same two subnets on every pyramid level. The class subnet consists of 4 conv and $k \times C$ outputs. The box subnet consists of 4 conv and $4k$ outputs. The network is trained with focal loss.

RetinaNet, Measured

RetinaNet, ResNet-101-FPN, 800px	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
COCO test-dev	37.8	57.5	40.8	20.2	41.1	49.2

What this settled

A one-stage detector beat every two-stage detector of its day, with **no architectural novelty** — the backbone, the FPN and the anchors are all from Lecture 7.

The lesson

Before concluding that an architecture is wrong for your problem, check whether it is being *trained* wrong.

Note $AP_S = 20.2$ against $AP_L = 49.2$: focal loss fixed the imbalance, not the resolution problem. Small objects are still hard.

Anchor-Free: Deleting the Reference Boxes

What Anchors Were Costing

The hyperparameters

- Scales, aspect ratios, and how many
- The positive and negative IoU thresholds
- Per-level assignment rules once you have an FPN

Tuned per dataset. A set that works on COCO can be actively wrong on aerial imagery or text.

The compute

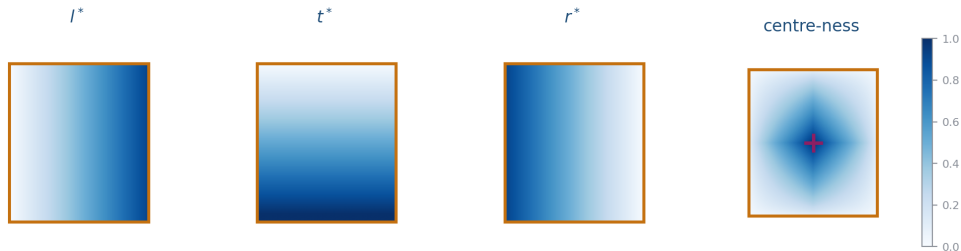
Every anchor needs an IoU against every ground-truth box, every iteration — and there are $\approx 10^5$ of them.

The question

If the network already sees the object at a location, why does it need a reference box to describe where the object is?

Fully Convolutional One-Stage - FCOS (Tian et al., 2019)

Treating object detection in a per-pixel prediction fashion, similar to semantic segmentation, rather than relying on predefined anchor boxes.



Every pixel inside the box regresses the four distances to its edges. Centre-ness = $\sqrt{\frac{\min(l, r)}{\max(l, r)} \cdot \frac{\min(t, b)}{\max(t, b)}}$ down-weights the sloppy boxes predicted from near the border.

The centerness score ranges from 0 to 1. During final post-processing (NMS), the classification score is multiplied by the centerness score.

FCOS: the Two Problems It Had to Solve

1. Ambiguity

A pixel inside two overlapping boxes has two sets of targets. FCOS assigns each level of the FPN a **range of regression distances**, so a large object and a small one are handled at different levels and rarely collide.

Where they still do, the smaller box wins.

2. Low-quality boxes far from the centre

A pixel near the border predicts a badly stretched box but can still be confident. The **centre-ness** branch predicts how central the pixel is, and multiplies the classification score at test time, so those boxes sink in the ranking.

FCOS	COCO AP
ResNet-50-FPN, baseline → with improvements	37.1 → 38.6
ResNeXt-64×4d-101-FPN	44.7

Anchors gone, with no accuracy lost — and centre-ness costs one extra 3×3 convolution.

CenterNet (Zhou et al., 2019): Objects as Points

Models each object as a single central point

objects are points



target heatmap (stride 4, shown upsampled)



The head predicts $C + 4$ maps:

class heatmap	C
local offset	2
object size w, h	2

σ comes from the box size, so a big object gets a broad peak and a small one a sharp peak.

At test time: keep every peak that is $\geq$ its 8 neighbours, take the top 100.

That max-pool IS the NMS.

The σ values are computed with the paper's rule: the radius is the largest displacement that still leaves $\text{IoU} \geq 0.7$ with the true box, so the target itself encodes how precise the centre has to be.

CenterNet, Measured — and the First NMS-Free Detector

Backbone	COCO AP	FPS
ResNet-18	28.1	142
DLA-34	37.4	52
Hourglass-104	40.3	14

Why it needs no NMS

Two detections of one object would be two peaks a few pixels apart. The Gaussian target makes that impossible to learn: there is exactly one maximum per object, and the 3×3 max-pool keeps exactly one.

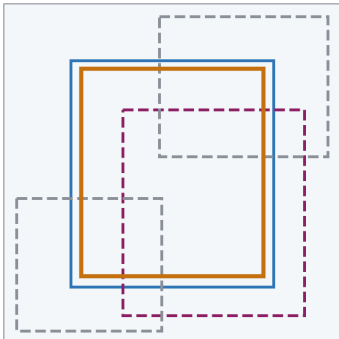
What else it gets for free

Swap the size head for depth and orientation, and you have 3D detection. Swap it for joint offsets, and you have pose estimation. The centre point is the only thing that had to be detected.

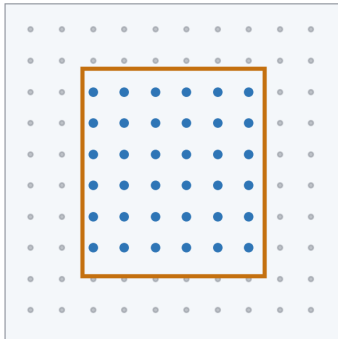
The limitation is inherent: two objects whose centres collide after downsampling become one peak. In crowded scenes that is a real cost.

Three Answers to One Question

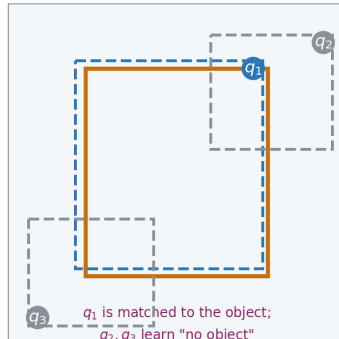
anchor-based: IoU thresholds



anchor-free: is the pixel inside?



set prediction: one global matching



Same object, three answers to "which prediction is responsible?" --- and each removal takes a set of hyperparameters with it.

DETR: Detection as Set Prediction

This topic began by saying detection returns a **set** of unknown size, and that every method fixes this by proposing a dense candidate set and pruning it.

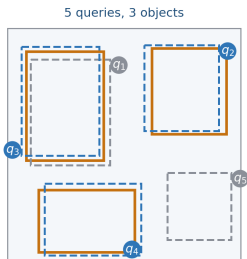
DETR does not. It predicts a fixed set of N slots *directly*, and makes the **loss** responsible for the fact that a set has no ordering.

What that buys

If the loss forbids two predictions from claiming the same object, nothing is left for NMS to remove. The duplicate problem is solved at training time, not at inference.

Every detector so far needed a rule for *which prediction is responsible*. DETR computes it, globally, at every step of training.

Bipartite Matching



one global assignment, not a greedy pass

q ₁	0.22	1.00	1.00
q ₂	1.00	0.24	1.00
q ₃	0.18	1.00	1.00
q ₄	1.00	1.00	0.23
q ₅	1.00	1.00	1.00
	gt 1	gt 2	gt 3

cost = 1 - IoU (the real loss adds class and L_1 terms)

Greedy would give q_3 to gt 1 and leave q_1 as a duplicate. The Hungarian algorithm minimises the TOTAL cost, so exactly one query is responsible for each object and the rest are trained to say "no object".

For *one* target $y_i = (c_i, b_i)$ with *one* prediction $\hat{y}_j$:

$$\mathcal{L}_{\text{match}}(y_i, \hat{y}_j) = -\mathbb{1}_{\{c_i \neq \emptyset\}} \hat{p}_j(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \left(\lambda_{L_1} \|b_i - \hat{b}_j\|_1 + \lambda_{\text{giou}} \mathcal{L}_{\text{giou}} \right).$$

i indexes objects, not images. Here $y = \{y_i\}_{i=1}^N$ is the image's objects padded with $\emptyset$ up to N , and $b_i \in [0, 1]^4$ is a **centre-size** box normalised by the image, not corner pixels

The Set Loss

Pad the ground truth with $\emptyset$ up to N , then find the permutation $\hat{\sigma}$ of the N predictions that minimises the total cost:

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}_N} \sum_{i=1}^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

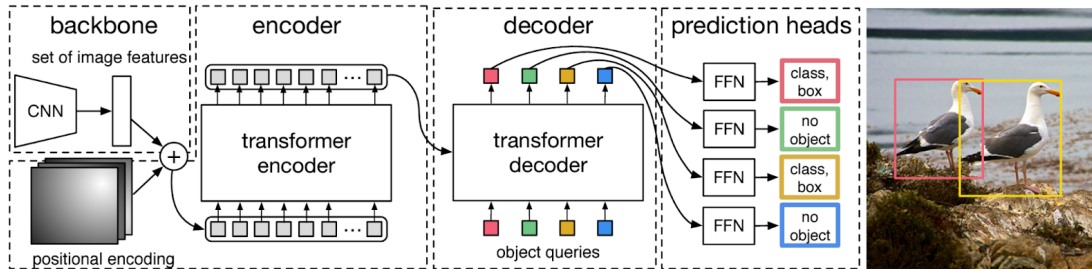
and train on that assignment:

$$\mathcal{L} = \sum_{i=1}^N \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} (\lambda_{L_1} \|b_i - \hat{b}_{\hat{\sigma}(i)}\|_1 + \lambda_{\text{giou}} \mathcal{L}_{\text{giou}}) \right]$$

Why the Hungarian algorithm. It finds the minimum-cost assignment in $O(N^3)$. Greedy matching would let a good prediction be stolen and leave a duplicate behind — exactly the failure mode we are trying to delete.

Why GloU as well as L_1 . L_1 on box coordinates scales with box size, so it over-weights large boxes; GloU is scale-invariant and, unlike IoU, still gives a gradient when the boxes do not overlap.

DETR, End to End



DETR uses a conventional CNN backbone to learn a 2D representation of an input image. The model flattens it and supplements it with a positional encoding before passing it into a transformer encoder. A transformer decoder then takes as input a small fixed number of learned positional embeddings, which we call object queries, and additionally attends to the encoder output. We pass each output embedding of the decoder to a shared feed forward network (FFN) that predicts either a detection (class and bounding box) or a “no object” class.

N learned embeddings, fed into the decoder as its input. There are no proposals and no anchors — a query is simply a slot that will be filled by at most one object.

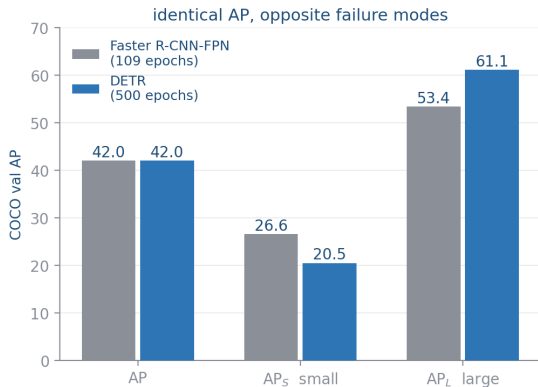
- $N = 100$, far more than the objects in any COCO image
- Queries specialise: each learns a preferred region and size range, an anchor the model chose for itself
- Self-attention between queries in the decoder is what lets them *avoid* each other — a query can see that another has already taken an object

Consequence: DETR cannot return more than N objects, and its accuracy falls on images with more. For COCO that never binds.

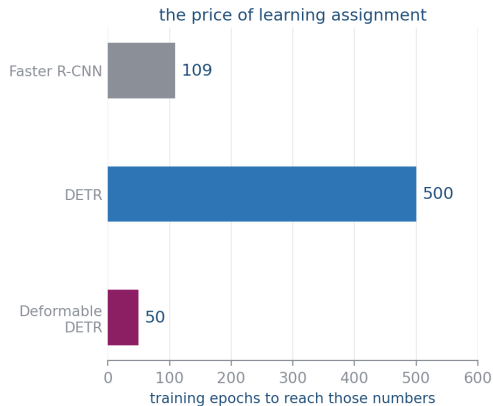
The connection worth naming

Anchors were a prior over *where boxes are*, written by hand. Object queries are the same prior, learned.

What It Cost



Numbers from Carion et al. (2020) and Zhu et al. (2021), ResNet-50 backbones.



Reading Those Two Charts

Why AP_S collapses

DETR predicts from a **single** coarse feature map — the encoder output at stride 32. There is no pyramid. Small objects have the same problem they had in Lecture 7's plain Faster R-CNN, and for the same reason.

Why AP_L jumps

Global self-attention from the first layer. A large object spanning the image is a single relationship for an attention layer and a long-range dependency for a convolution.

Why 500 epochs

The matching is unstable early: the same object is claimed by different queries from one step to the next, so the queries take a long time to specialise. **Deformable DETR** (Zhu et al., 2021) attends to a few learned sampling points instead of all of them, adds multi-scale features, and reaches **46.9 AP in 50 epochs** — with AP_S rising from 20.5 to 26.4.

What DETR Proved

The real contribution

Not the transformer. The **set loss**: if the training objective itself forbids duplicates, the entire post-processing stage becomes unnecessary.

That idea is separable from the architecture. As we are about to see, it did separate.

What it also demonstrated

Detection, segmentation and panoptic output all follow from the same decoder with a different head. A general architecture, in the sense Lecture 6 meant.

Hand-designed components removed: proposals, anchors, assignment heuristics, and NMS. All four.

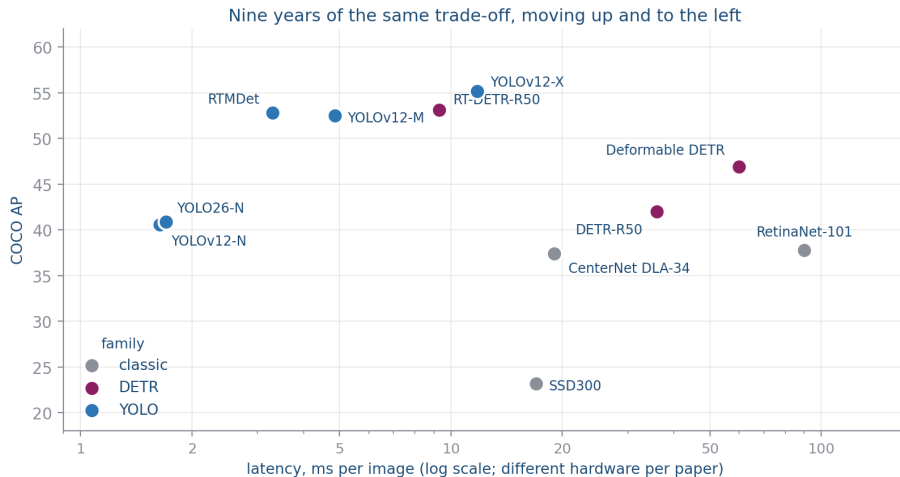
What You Should Actually Use

The Ledger

	region proposals	anchors	IoU assignment thresholds	sampling / reweighting	NMS
Faster R-CNN	needs it	needs it	needs it	needs it	needs it
RetinaNet	gone	needs it	needs it	needs it	needs it
FCOS / CenterNet	gone	gone	gone	needs it	needs it
DETR	gone	gone	gone	gone	gone

Every column that empties is a set of hyperparameters you no longer tune --- and a set of failure modes you no longer have.

The Frontier, Today



Latency is measured on different hardware by different authors, so compare the *shape* of the frontier, not neighbouring points.

Where the Two Families Ended Up

DETR became fast

RT-DETR (2023) reaches **53.1 AP at 108 FPS** with a ResNet-50 by making the encoder cheap — and argues that removing NMS is itself a speed win, because NMS latency varies with the number of detections.

YOLO became NMS-free

The recent YOLO releases adopt one-to-one matching and drop NMS at inference. The idea DETR introduced now ships inside the family DETR was competing with.

The honest summary of 2026

The architectural argument is over. Both families are transformer-influenced, both are NMS-free, and the choice between them is now an empirical question about *your* data.

For Your Final Project

Situation	Reach for	Licence
A strong default, permissively licensed	RF-DETR	Apache 2.0
Maximum throughput per frame	RTMDet	MIT
Fastest path to a working baseline	a recent YOLO	AGPL-3.0
No labels yet, exploring feasibility	YOLO-World, Grounding DINO	varies
Understanding <i>why</i> it fails	train RetinaNet or FCOS yourself	—

Check the licence before you commit

AGPL-3.0 obliges you to release your source if you serve the model over a network. For coursework that is fine; for anything you intend to ship, it is a decision, not a detail.

The method that beats the table

Label your data, then train two architectures side by side and let your own validation set decide. Published COCO rankings rarely survive contact with a specific domain.

Nine Years, One Component at a Time

Each step deletes a hand-designed component --- and pays for it in data, training time, or both.



Read it as a single argument: every hand-designed component in a detector turned out to be something a model could learn, given enough data and enough training.

And read the cost column too. RetinaNet needed a new loss; FCOS needed a new branch; DETR needed 500 epochs. Nothing was free.

- ➊ Removing the proposal stage exposes a $\approx 2000:1$ foreground–background imbalance — we counted it
- ➋ YOLO and SSD patched it with a constant and with hard negative mining; both worked, neither explained anything
- ➌ Focal loss reweights instead of resampling, moving the loss share onto the objects by roughly $10\times$
- ➍ RetinaNet: a one-stage detector that won on *loss design alone*
- ➎ FCOS and CenterNet delete anchors: regress distances from a pixel, or find a peak. Centre-ness and the Gaussian target replace IoU rules
- ➏ DETR deletes assignment *and* NMS with a Hungarian-matched set loss — and pays 500 epochs for it
- ➐ Deformable DETR pays 50 instead, by restoring multi-scale features
- ➑ In 2026 both families are NMS-free; choose on your own data, and read the licence

- Redmon et al., “You Only Look Once”, *CVPR* 2016
- Liu et al., “SSD: Single Shot MultiBox Detector”, *ECCV* 2016
- Lin et al., “Focal Loss for Dense Object Detection”, *ICCV* 2017 — read section 3 twice
- Tian et al., “FCOS: Fully Convolutional One-Stage Object Detection”, *ICCV* 2019
- Zhou et al., “Objects as Points”, 2019 — CenterNet
- Carion et al., “End-to-End Object Detection with Transformers”, *ECCV* 2020 — DETR
- Zhu et al., “Deformable DETR”, *ICLR* 2021
- Zhao et al., “DETRs Beat YOLOs on Real-time Object Detection”, 2023

Code worth reading: `torchvision.ops.sigmoid_focal_loss` is six lines and matches the equation exactly; `detr/models/matcher.py` is about forty and is the whole idea.

lecture08_handson.ipynb: build the three losses that define this lecture.

- 1 Focal loss from scratch, and the gradient it produces
- 2 Reproduce the loss-share figure on your own simulated anchors
- 3 The bias initialisation: show that training diverges without it
- 4 FCOS targets: l^* , t^* , r^* , b^* and centre-ness, vectorised
- 5 CenterNet's Gaussian radius rule, and why σ depends on size
- 6 Hungarian matching against greedy, on the same cost matrix
- 7 A DETR-style set loss, end to end on toy data
- 8 Measure NMS latency against detection count — and explain RT-DETR's claim